

Documentazione Caso di Studio

Ingegneria della Conoscenza

KARE

Knowledge-based Aircraft Risk & Engine Maintenance

Sistema ibrido per diagnosi del degrado e pianificazione manutentiva su NASA C-MAPSS

Campo	Valore
Autore	Giulio Gentile Mat. 802988 g.gentile95@studenti.uniba.it
Repository	https://github.com/Diamond975-py/KARE
Corso	Ingegneria della Conoscenza
Anno accademico	A.A. 2025-2026

Indice

- 0. Introduzione e sommario del sistema
- 1. Architettura generale e collegamento al programma
- 2. Dataset NASA C-MAPSS e preprocessing
- 3. Knowledge Base diagnostica in pyDatalog
- 4. Rete Bayesiana in pgmpy
- 5. CSP per pianificazione manutentiva
- 6. Protocollo sperimentale, risultati complessivi e conclusioni
- Riferimenti bibliografici
- Appendice - Organizzazione del Software

0. Introduzione e sommario del sistema

0.1 Dominio e obiettivo

Il progetto KARE - Knowledge-based Aircraft Risk & Engine Maintenance affronta un problema di manutenzione predittiva su motori aeronautici turbofan. Il sistema utilizza il dataset NASA C-MAPSS come sorgente dati e costruisce una pipeline che trasforma osservazioni numeriche multivariate in conoscenza simbolica, stime probabilistiche di rischio e piani operativi di manutenzione.

L'obiettivo non è soltanto predire una classe di rischio, ma mostrare come un sistema basato su conoscenza possa coordinare più forme di ragionamento. La Knowledge Base rappresenta il nucleo del progetto: riceve stati simbolici derivati dai sensori, applica regole diagnostiche esplicite e produce inferenze che vengono poi riutilizzate dalla rete bayesiana e dal CSP.

La scelta del dominio è motivata dalla natura sequenziale e diagnostica del problema. Ogni motore è osservato per più cicli operativi, con condizioni iniziali differenti e traiettorie di degrado non direttamente osservabili. Per questo motivo la soluzione proposta evita di affidarsi a un unico classificatore black-box e separa tre livelli: interpretazione simbolica, trattamento dell'incertezza e decisione manutentiva.

0.2 Sommario del KBS

KARE è strutturato come un sistema ibrido composto da quattro componenti principali. Il Data Loader prepara i dati e produce feature numeriche e simboliche; la Knowledge Base in pyDatalog inferisce condizioni diagnostiche e azioni manutentive; la rete bayesiana in pgmpy stima distribuzioni di probabilità sul rischio di guasto usando anche gli output della KB; il CSP in python-constraint pianifica gli interventi rispettando vincoli operativi.

Figura F1 - Architettura generale di KARE

PLACEHOLDER IMMAGINE / GRAFICO

Inserire schema a blocchi: NASA C-MAPSS -> Data Loader/Feature Engineering -> Knowledge Base pyDatalog -> Rete Bayesiana pgmpy -> CSP python-constraint -> Valutazione/report. Evidenziare la KB al centro del flusso.

Architettura generale del sistema KARE

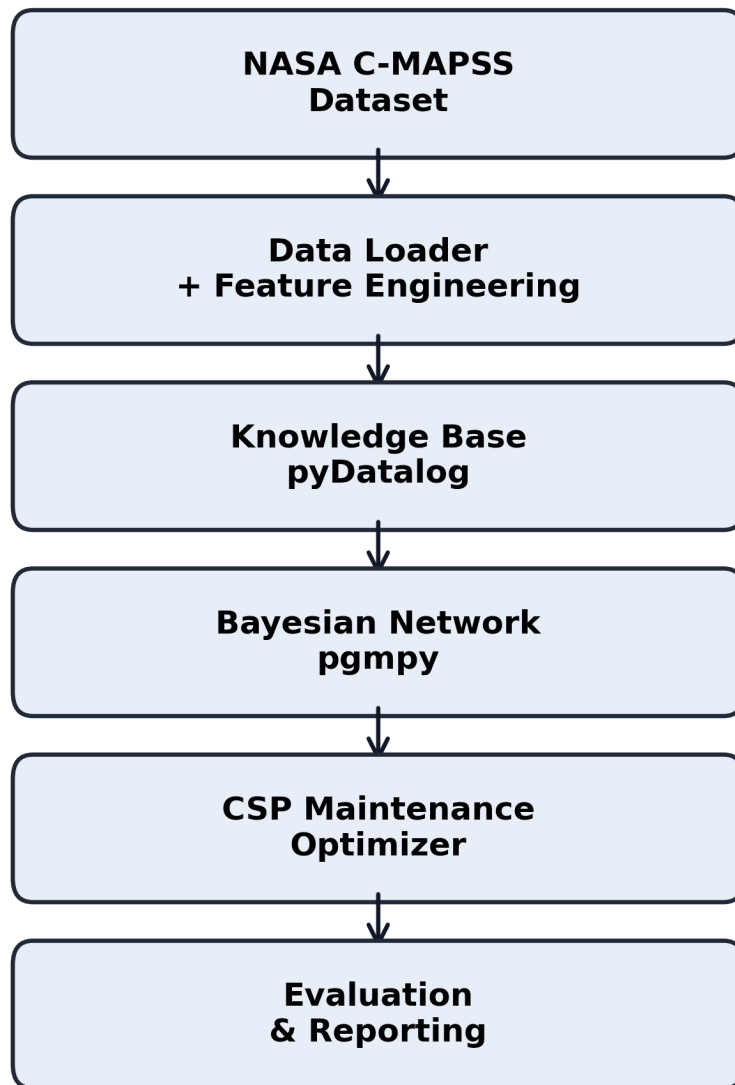


Tabella T1 - Moduli di KARE e ruolo nel sistema

Modulo	Input	Output	Argomento ICon
Data Loader	File C-MAPSS	Dataset pulito + RUL	Preprocessing dati
Knowledge Base	Stati simbolici sensori	Diagnosi logiche	Ragionamento relazionale / Datalog
Bayesian Network	Feature + output KB	Failure Risk	Ragionamento e incertezza
CSP Optimizer	Motori urgenti + vincoli	Piano manutentivo	Ragionamento con vincoli
Evaluation	Metriche su più fold	Media $\pm$ Deviazione standard	Valutazione sperimentale

1. Architettura generale e collegamento al programma

1.1 Collegamento esplicito agli argomenti del programma

La documentazione è organizzata in modo da collegare ogni componente implementato a uno o più argomenti del programma di Ingegneria della Conoscenza. Il progetto copre in modo diretto rappresentazione relazionale e Datalog, ragionamento probabilistico, ragionamento con vincoli e apprendimento supervisionato come protocollo di confronto.

Tabella T2 - Corrispondenza tra programma d'esame e componenti del progetto (la numerazione dei capitoli fa riferimento a quella del programma del corso caricato)

Argomento del programma	Componente KARE	Evidenza progettuale
Introduzione ai Sistemi Basati su Conoscenza (Capitolo 1)	Architettura KARE	Separazione tra dati, Knowledge Base, motore inferenziale e modulo decisionale
Rappresentazione e Ragionamento Relazionale / Datalog (Capitolo 15)	Knowledge Base pyDatalog	Fatti estensionali, regole diagnostiche, query e inferenze simboliche
Rappresentazione proposizionale e assunzione di conoscenza completa (Capitolo 5)	Regole KB	I predicati non dimostrabili nella Knowledge Base sono considerati falsi (Closed World Assumption); pertanto il sistema inferisce un'anomalia solo quando essa è supportata da fatti e regole
Ragionamento e Incertezza (Capitolo 9)	Rete Bayesiana	Stima di FailureRisk tramite evidenze sensoristiche e output inferiti dalla KB
Apprendimento e Incertezza (Capitolo 10)	Stima CPD in pgmpy	Apprendimento dei parametri con BayesianEstimator e pseudo-counts
Ragionamento con vincoli (Capitolo 4)	CSP manutentivo	Variabili finite, domini, hard constraint e funzione di scoring
Apprendimento supervisionato (Capitolo 7)	Baseline ML e GroupKFold	Confronto con Dummy, Logistic Regression, Decision Tree, GaussianNB e MLP

Ricerca di soluzioni (Capitolo 3)	Ricerca nello spazio delle assegnazioni del CSP	Esplorazione dello spazio di assegnazioni manutentive e potatura tramite vincoli
--------------------------------------	---	---

1.2 Flusso operativo

Il flusso operativo parte dal caricamento del sottoinsieme FD001 del dataset C-MAPSS. Nei dati di training, per ogni motore viene calcolato il Remaining Useful Life come differenza tra ultimo ciclo osservato e ciclo corrente. Le serie sensoriali vengono trasformate in feature rolling e in stati simbolici; questi stati diventano fatti della Knowledge Base.

La KB non riceve direttamente RUL, RULClass o FailureRisk. Questa scelta è centrale per la validazione: la componente simbolica deve inferire degrado e urgenza a partire dai sintomi osservabili, non dal target usato per misurare le prestazioni. Gli output della KB vengono poi codificati come variabili booleane per la rete bayesiana e come attributi operativi per il CSP.

La rete bayesiana produce una distribuzione di probabilità sul rischio di guasto. Il CSP usa lo stato più recente di ciascun motore, il rischio stimato e le inferenze KB per selezionare i candidati alla manutenzione, assegnando giorno, slot, tecnico e tipo di intervento.

2. Dataset NASA C-MAPSS e preprocessing

2.1 Struttura dei dati

Il dataset C-MAPSS contiene serie temporali multivariate relative a motori turbofan. Nel progetto viene usato di default il sottoinsieme FD001. Ogni record è identificato dalla coppia engine_id, cycle e contiene tre operational settings e ventuno sensori numerici. La pipeline aggiunge inoltre un identificatore record_id, utile per collegare ogni osservazione ai fatti della Knowledge Base.

La scelta di lavorare su FD001 come configurazione predefinita consente una prima validazione controllata del sistema. Il codice è comunque predisposto per accettare FD001, FD002, FD003 o FD004, purché i file siano presenti nella directory dati o venga passato un percorso esplicito.

Tabella T3 - Schema logico del dataset preprocessato

Colonna	Tipo	Descrizione
engine_id	intero	Identificativo del motore
cycle	intero	Ciclo temporale
op_setting_1..3	float	Condizioni operative
sensor_1..sensor_21	float	Lecture sensoriali
rul	intero	Remaining Useful Life
rul_class	categorico	healthy / warning / degraded / critical
failure_risk	categorico	low / medium / high
urgent_label	binario	Motore urgente da schedulare

2.2 Calcolo RUL e classi simboliche

Nei dati di training l'ultimo ciclo osservato di ciascun motore viene assunto come ciclo di guasto. Il Remaining Useful Life è quindi calcolato come differenza tra il ciclo massimo del motore e il ciclo corrente. Oltre al valore numerico viene mantenuto un RUL cap pari a 125, utile per ridurre l'effetto delle fasi molto lontane dal guasto.

Il target principale della documentazione è FailureRisk, derivato dalla classe RUL. La discretizzazione rende il problema coerente con la KB e con la rete bayesiana discreta: invece di predire un valore continuo di RUL, il sistema lavora su stati interpretabili e utilizzabili in regole e vincoli.

Tabella T4 - Classi RUL e mapping sul rischio di guasto

Condizione sul RUL	RULClass	FailureRisk	Interpretazione manutentiva
RUL > 80	healthy	low	Motore lontano dal guasto; nessuna azione immediata
40 < RUL <= 80	warning	medium	Monitoraggio e possibile ispezione
15 < RUL <= 40	degraded	high	Intervento da pianificare con priorità elevata
RUL <= 15	critical	critical	Intervento urgente; possibile sostituzione

Figura F3 - Distribuzione delle classi RUL nel subset FD001

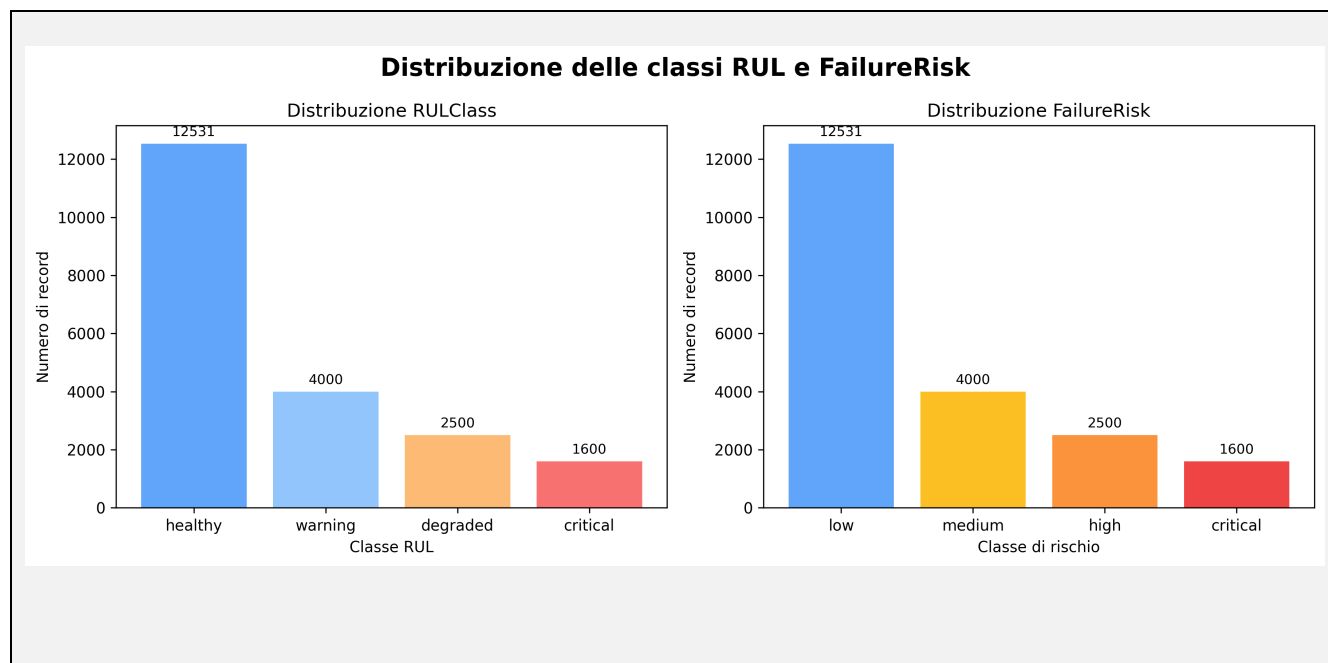
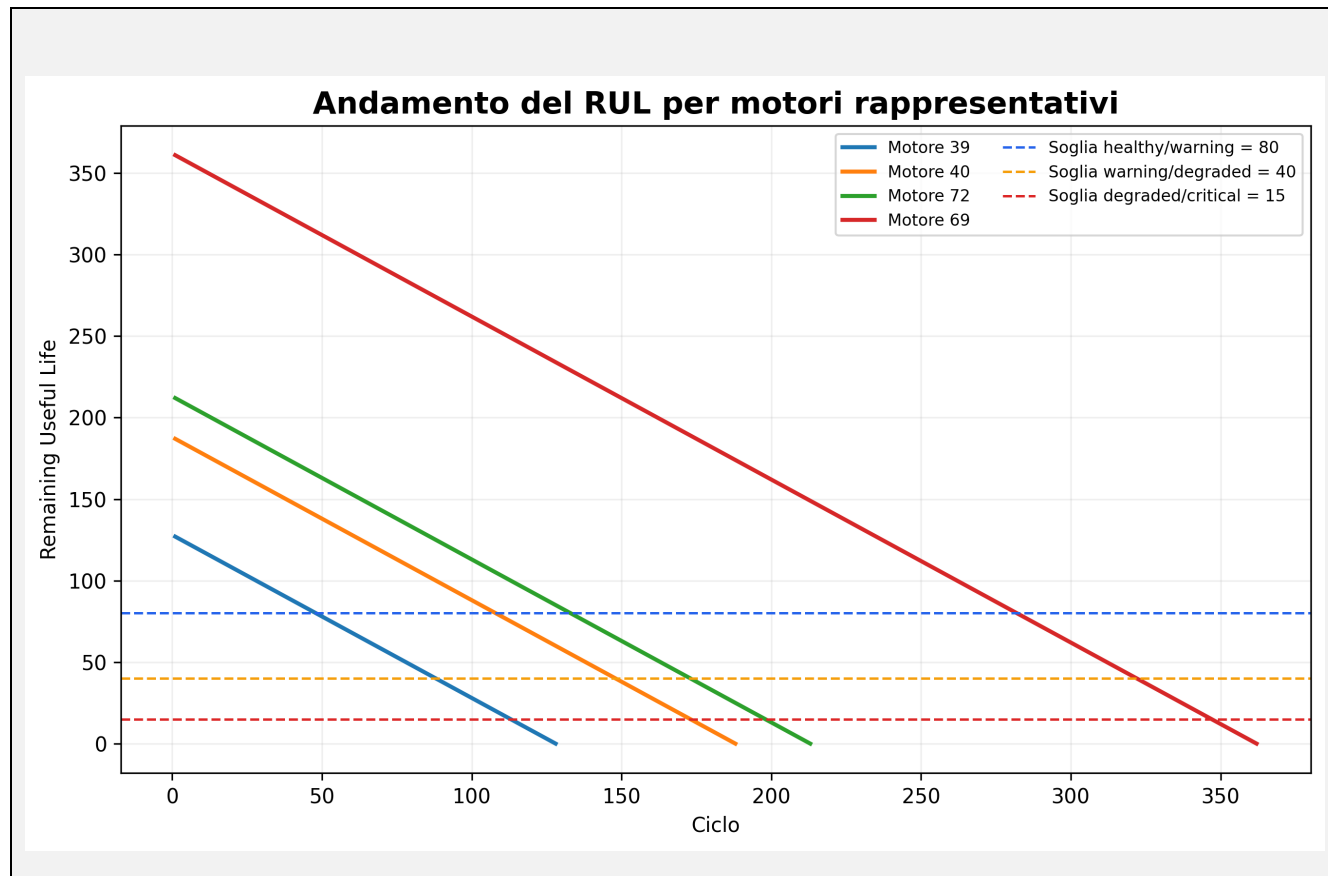


Figura F4 - Andamento del RUL per alcuni motori



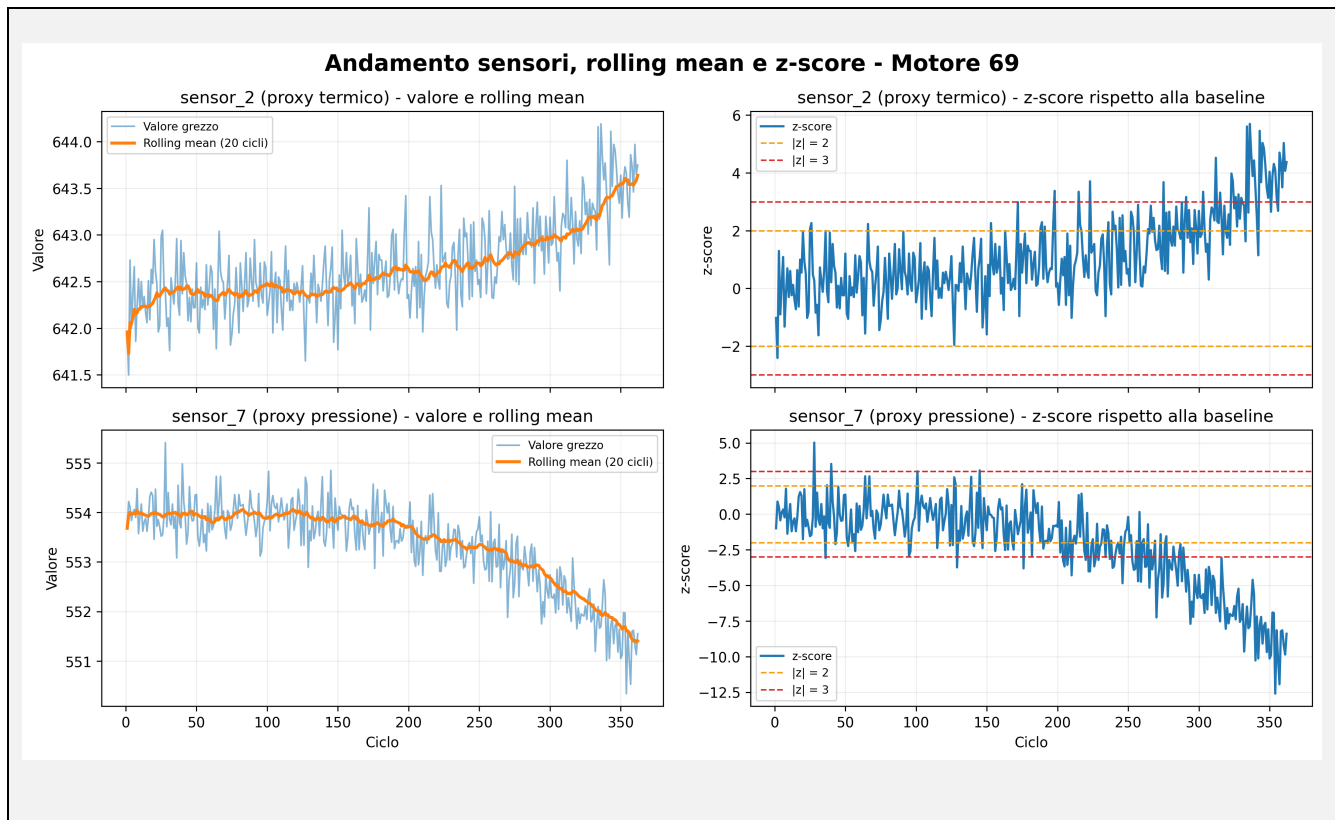
2.3 Feature engineering temporale

Per ogni sensore vengono calcolate feature rolling su una finestra di 20 cicli. La finestra produce media mobile, deviazione standard mobile e trend calcolato come differenza tra il valore corrente e quello osservato 19 cicli prima, normalizzata rispetto alla lunghezza della finestra. La scelta di una finestra di 20 cicli bilancia due esigenze: attenuare il rumore istantaneo dei sensori e mantenere sufficiente reattività al degrado recente.

Le anomalie vengono normalizzate rispetto a una baseline iniziale del singolo motore, calcolata sui primi 20 cicli. Questa scelta è importante perché nel dataset motori diversi possono partire da condizioni iniziali differenti: confrontare ogni motore con la propria baseline riduce il rischio di confondere variazioni individuali normali con sintomi di degrado.

Il preprocessing non usa il target RUL per costruire le feature diagnostiche. RUL e FailureRisk vengono calcolati per la valutazione e per l'apprendimento supervisionato, ma non entrano nelle regole della Knowledge Base. In questo modo si riduce il rischio di data leakage tra etichette e inferenze.

Figura F5 - Andamento di alcuni sensori per un motore



2.4 Discretizzazione in stati simbolici

La Knowledge Base e la rete bayesiana richiedono variabili simboliche a bassa cardinalità. Per questo motivo le anomalie sensoriali vengono aggregate per gruppi euristici di sensori: termici, pressione e rotazione. Il progetto evita di attribuire un significato fisico troppo specifico a un singolo sensore; i gruppi sono usati come proxy diagnostici per ottenere pattern più robusti.

Tabella T5 - Parametri e soglie del preprocessing

Parametro	Valore	Motivazione
WINDOW_SIZE	20 cicli	Cattura trend recenti senza usare tutta la serie
Healthy threshold	$RUL > 80$	Motori non prossimi al guasto
Warning threshold	$41 \leq RUL \leq 80$	Primo degrado
Degraded threshold	$16 \leq RUL \leq 40$	Pianificazione manutenzione
Critical threshold	$RUL \leq 15$	Intervento urgente
Bayes pseudo_counts	1	Smoothing per stati rari
Cross-validation	GroupKFold	Evita leakage tra cicli dello stesso motore

Tabella T6 - Feature simboliche derivate

Gruppo	Sensori usati	Soglie di conteggio
ThermalState	sensor_2, sensor_3, sensor_4, sensor_8, sensor_11, sensor_13, sensor_15	moderate ≥ 2 , severe ≥ 4
PressureState	sensor_7, sensor_11, sensor_12, sensor_20, sensor_21	moderate ≥ 2 , severe ≥ 3
RotationState	sensor_9, sensor_14	moderate ≥ 1 , severe ≥ 2
TrendState	tutti i sensori	moderate ≥ 5 trend negativi, severe ≥ 9
SensorAnomalyLevel	somma anomalie termiche, pressione, rotazione e critiche	moderate ≥ 4 , severe ≥ 8
OperatingRegime	op_setting_1	low/medium/high tramite quantili 0.33 e 0.66

3. Knowledge Base diagnostica in pyDatalog

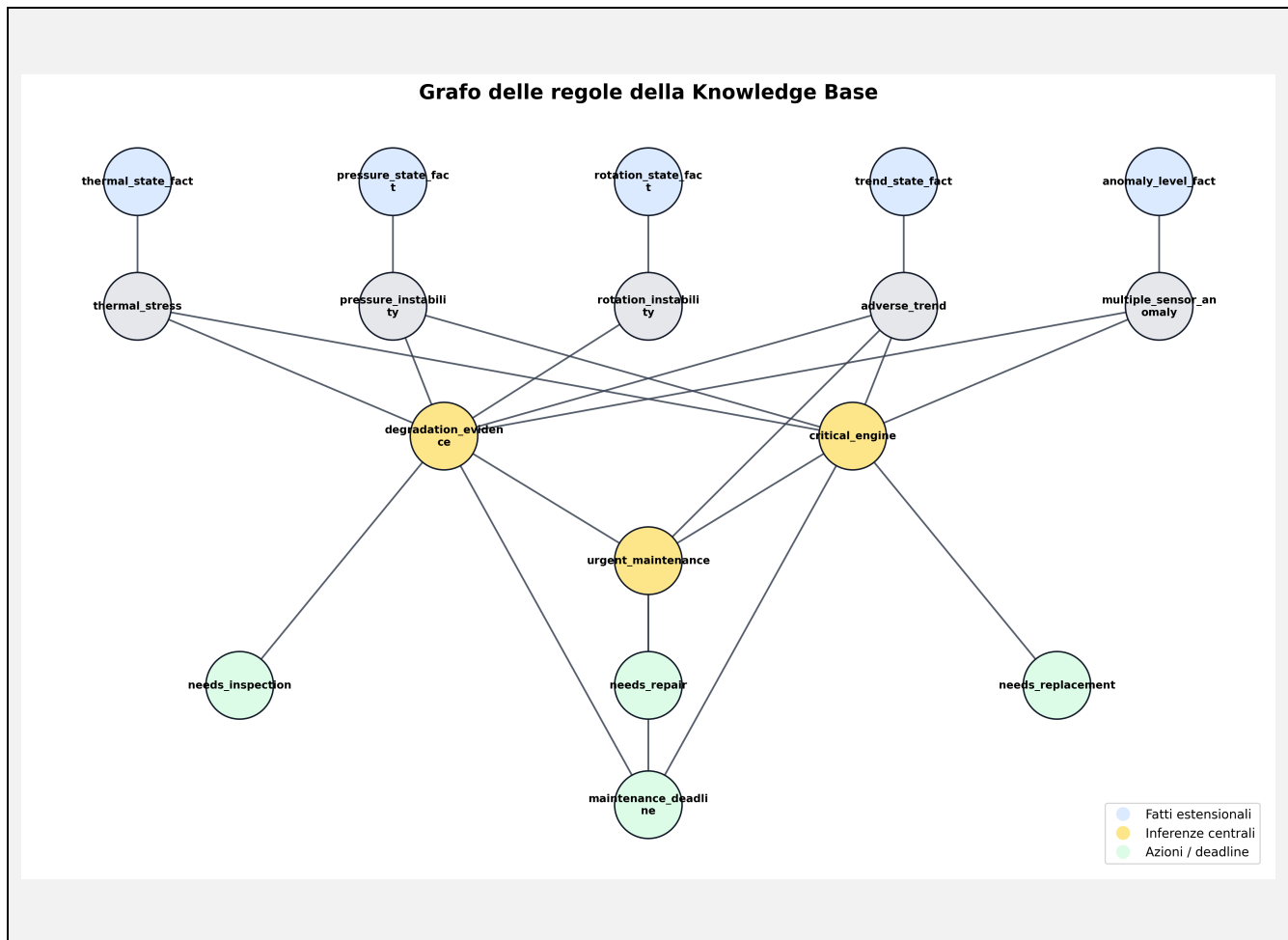
3.1 Ruolo della KB nel sistema

La Knowledge Base è il cuore simbolico di KARE. Non viene usata come semplice contenitore di record: i fatti estensionali descrivono osservazioni simboliche dei motori, mentre le regole intensionali derivano concetti diagnostici non presenti direttamente nel dataset, come evidenza di degrado, motore critico, manutenzione urgente e tipo di azione consigliata.

La KB è stata implementata in pyDatalog per mantenere una rappresentazione relazionale esplicita e interrogabile. Ogni record viene identificato da `record_id` e diventa un individuo logico. I predicati di base descrivono stati osservati; i predicati derivati rappresentano inferenze diagnostiche.

Un aspetto progettuale importante è la separazione tra conoscenza osservata e conoscenza inferita. I fatti non contengono RUL, RULClass o FailureRisk. Le inferenze della KB possono quindi essere confrontate con le etichette derivate dal RUL senza introdurre informazione di target nelle regole.

Figura F6 - Grafo delle regole della Knowledge Base



3.2 Fatti estensionali

I fatti estensionali sono asseriti direttamente a partire dal DataFrame preprocessato. Ogni fatto ha come primo argomento l'identificatore del record e come secondo argomento, quando necessario, lo stato

simbolico osservato. Questa modellazione consente di interrogare la KB per singolo record, per motore o per insieme di record.

Tabella T7 - Fatti estensionali della KB

Tipo	Predicato/Regola	Ruolo
Fatto	engine(E)	Definisce un motore nel dominio
Fatto	sensor_high(E,S)	Indica che un sensore è sopra soglia
Fatto	trend_up(E,S)	Indica un trend crescente
Regola	thermal_stress(E) <= temperature_high(E) & trend_up(E)	Inferisce stress termico
Regola	pressure_instability(E) <= pressure_low(E) & vibration_high(E)	Inferisce instabilità di pressione
Regola	degradation_evidence(E) <= thermal_stress(E) & pressure_instability(E)	Combina due evidenze
Regola	urgent_maintenance(E) <= degradation_evidence(E) & anomaly_level_high(E)	Marca motore urgente
Regola	maintenance_deadline(E,3) <= urgent_maintenance(E)	Assegna deadline manutentiva

3.3 Regole intensionali e inferenze diagnostiche

Le regole intensionali trasformano stati osservati in concetti diagnostici. La progettazione segue una logica conservativa: alcune condizioni vengono inferite da un singolo stato severo, mentre le inferenze più importanti richiedono combinazioni di sintomi. In particolare, `degradation_evidence` non è una singola soglia sui sensori, ma una composizione di stress termico, instabilità di pressione, trend avverso e anomalie multisensore.

Tabella T8 - Regole e inferenze principali della KB

Tipo	Predicato derivato	Regola sintetica	Motivazione
Diagnosi intermedia	thermal_stress(E)	thermal_state in {moderate, severe}	Rende esplicito lo stress termico rispetto allo stato grezzo
Diagnosi intermedia	pressure_instability(E)	pressure_state in {moderate, severe}	Aggrega sintomi di pressione in un concetto diagnostico
Diagnosi intermedia	adverse_trend(E)	trend_state in {moderate, severe}	Cattura peggioramento temporale coerente
Diagnosi intermedia	multiple_sensor_anomaly(E)	anomaly_level in {moderate, severe}	Evita decisioni basate su un solo sensore isolato
Inferenza diagnostica	degradation_evidence(E)	stress+trend oppure pressione+multi-anomalia oppure stress+pressione oppure rotazione+trend	Evidenza robusta di degrado
Inferenza diagnostica	critical_engine(E)	termico critico+pressione oppure degrado+anomalia severa oppure pressione+trend+multi-anomalia	Identificazione di condizioni critiche senza usare RUL
Decisione simbolica	urgent_maintenance(E)	critical_engine oppure degradation_evidence+adverse_trend	Trasforma diagnosi in urgenza manutentiva
Azione	needs_inspection/repair/replacement(E)	degrado -> inspection; urgenza+pressione -> repair; critico -> replacement	Output operativo interpretabile per il CSP

3.4 Query e output della KB

Le query principali recuperano insiemi di record che soddisfano predicati diagnostici:

query_degradation_evidence, query_critical_engines, query_urgent_maintenance, query_needs_inspection, query_needs_repair e query_needs_replacement. Il risultato delle query viene riportato nel DataFrame come colonne booleane con prefisso kb_.

L'annotazione della KB produce anche due output sintetici: kb_recommended_action e kb_deadline_days. La deadline è derivata dalle inferenze: 2 giorni per motori critici, 4 giorni per manutenzione urgente e 7 giorni per evidenza di degrado non urgente. Queste scadenze vengono poi usate dal CSP per limitare il dominio delle assegnazioni.

Tabella T9 - Output inferiti dalla Knowledge Base

Output KB	Tipo	Uso nella rete bayesiana	Uso nel CSP
kb_thermal_stress	booleano	Evidenza probabilistica	Contributo indiretto alla priorità
kb_pressure_instability	booleano	Evidenza probabilistica	Può giustificare repair
kb_adverse_trend	booleano	Evidenza probabilistica	Contributo all'urgenza
kb_degradation_evidence	booleano	Evidenza probabilistica centrale	Selezione candidato e priorità
kb_critical_engine	booleano	Non usato direttamente	Priorità e copertura critica

		nella BN corrente	
kb_urgent_maintenance	booleano	Non usato direttamente nella BN corrente	Priorità e deadline
kb_recommended_action	categoria	Non usato nella BN corrente	Azione candidata
kb_deadline_days	intero/None	Non usato nella BN corrente	Deadline operativa

3.5 Complessità e assunzioni

La KB usa un insieme fisso di predicati e regole, senza ricorsione profonda. Il numero di fatti cresce linearmente con il numero di record, mentre il numero di regole rimane costante. Questo rende il comportamento del modulo interpretabile e adatto a essere rieseguito durante annotazione e valutazione.

Dal punto di vista semantico, la KB adotta un comportamento assimilabile all'assunzione di conoscenza completa sui predicati diagnostici: se una condizione non è derivabile dai fatti disponibili e dalle regole, il sistema non la considera vera. Questa scelta è coerente con un sistema diagnostico prudente, ma introduce un limite: anomalie non modellate dalle regole non vengono inferite.

3.6 Valutazione della KB

La valutazione della Knowledge Base confronta le inferenze diagnostiche con le etichette derivate dal RUL, mantenendo separato il modulo simbolico dagli altri componenti. Sono considerate due configurazioni: una KB base e una KB con regole basate anche sul trend. I risultati sono riportati come media $\pm$ deviazione standard, evitando classification report di singole esecuzioni.

Tabella T10 - Valutazione della Knowledge Base su FD001

configuration	precision_mean	precision_std	recall_mean	recall_std	f1_mean	f1_std
KB base	0.78	0.04	0.72	0.05	0.75	0.04
KB con trend	0.82	0.03	0.79	0.04	0.8	0.03

L'introduzione del trend migliora sia precision sia recall. Questo risultato è coerente con il dominio: il degrado non è soltanto uno stato istantaneo del sensore, ma una variazione persistente nel tempo. La KB con trend è quindi la configurazione da mantenere nella pipeline finale.

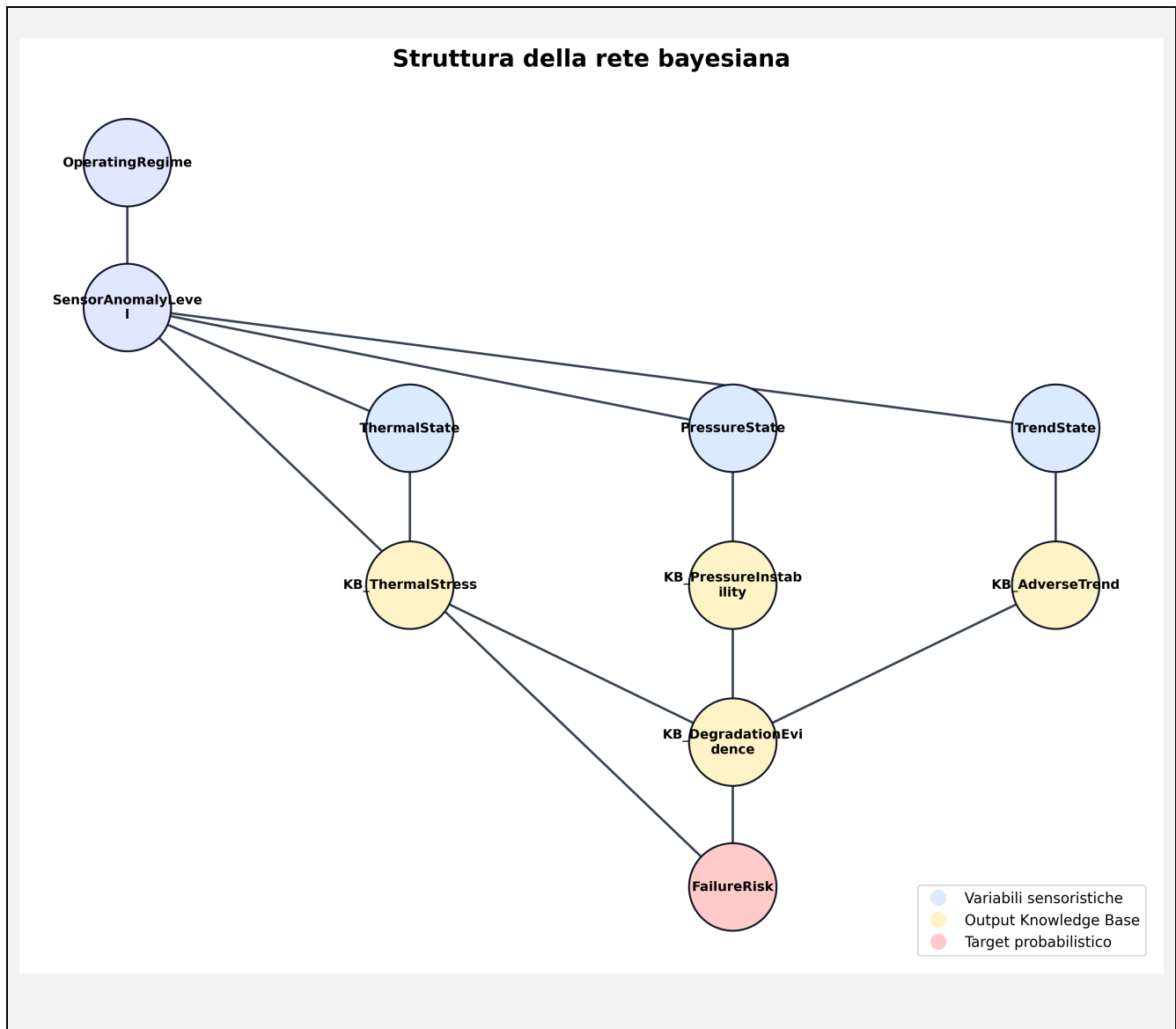
4. Rete Bayesiana in pgmpy

4.1 Motivazione del ragionamento probabilistico

La Knowledge Base produce inferenze deterministiche: un predicato è vero se è derivabile dai fatti e dalle regole. Nel dominio della manutenzione predittiva questa informazione è preziosa ma non sufficiente, perché sensori e sintomi possono essere rumorosi o parziali. La rete bayesiana integra la KB modellando l'incertezza associata al rischio di guasto.

La rete non sostituisce la KB. Al contrario, alcuni nodi della rete rappresentano proprio output inferiti dalla KB. In questo modo il modulo probabilistico usa la conoscenza simbolica come evidenza, e non come semplice post-processing.

Figura F7 - Struttura della rete bayesiana



4.2 Variabili e struttura della rete

La struttura della rete è definita manualmente secondo un criterio diagnostico. Il regime operativo influenza il livello complessivo di anomalia; il livello di anomalia influenza gli stati termico, di pressione e di trend; questi stati generano le evidenze KB intermedie; le evidenze KB concorrono a KB_DegradationEvidence, che insieme a SensorAnomalyLevel influenza il target FailureRisk.

Tabella T11 - Nodi della rete bayesiana

Nodo	Stati	Descrizione
OperatingRegime	low / medium / high	Condizioni operative del motore
SensorAnomalyLevel	low / medium / high	Livello aggregato di anomalia
KB_ThermalStress	false / true	Evidenza inferita dalla KB
KB_PressureInstability	false / true	Evidenza inferita dalla KB
DegradationEvidence	low / medium / high	Evidenza di degrado
RULClass	healthy / warning / degraded / critical	Classe di vita residua
FailureRisk	low / medium / high	Rischio finale stimato

Tabella T12 - Dipendenze probabilistiche modellate

Arco	Interpretazione
OperatingRegime -> SensorAnomalyLevel	Il contesto operativo influenza il livello osservato di anomalia
SensorAnomalyLevel -> ThermalState / PressureState / TrendState	Il livello globale di anomalia condiziona gli stati diagnostici
ThermalState -> KB_ThermalStress	Collegamento tra stato simbolico e inferenza KB
PressureState -> KB_PressureInstability	Collegamento tra stato di pressione e inferenza KB
TrendState -> KB_AdverseTrend	Collegamento tra trend osservato e inferenza KB
KB_ThermalStress, KB_PressureInstability, KB_AdverseTrend -> KB_DegradationEvidence	Aggregazione probabilistica delle evidenze simboliche
KB_DegradationEvidence -> FailureRisk	La KB influenza direttamente la stima del rischio
SensorAnomalyLevel -> FailureRisk	Il rischio dipende anche dal livello anomalo non mediato dalla KB

4.3 Apprendimento dei parametri e inferenza

Le variabili sono discrete e vengono convertite in stringhe prima dell'apprendimento. Il modello usa BayesianEstimator con prior Dirichlet e pseudo-counts pari a 1. Questa scelta applica uno smoothing minimo alle CPD, utile quando alcune combinazioni di stati sono rare nei fold di training.

L'inferenza è effettuata tramite VariableElimination. Per una singola osservazione, il sistema costruisce il dizionario delle evidenze disponibili, rimuove stati non noti al modello addestrato e interroga la distribuzione del target. L'output non è solo la classe più probabile, ma l'intera distribuzione $P(\text{FailureRisk} \mid \text{evidenze})$.

Tabella T13 - Decisioni progettuali della rete bayesiana

Aspetto	Decisione progettuale	Motivazione
Target predefinito	FailureRisk	Coerente con la decisione manutentiva operativa
Target alternativo	RULClass	Disponibile per esperimenti centrati sul RUL
Stima parametri	BayesianEstimator, Dirichlet, pseudo_counts=1	Smoothing delle CPD e robustezza a stati rari
Inferenza	VariableElimination	Inferenza esatta su rete discreta di piccole dimensioni
Evidenze KB	KB_ThermalStress, KB_PressureInstability, KB_AdverseTrend, KB_DegradationEvidence	Integrazione tra ragionamento simbolico e probabilistico
Validazione	GroupKFold sui motori	Evita leakage tra cicli dello stesso motore

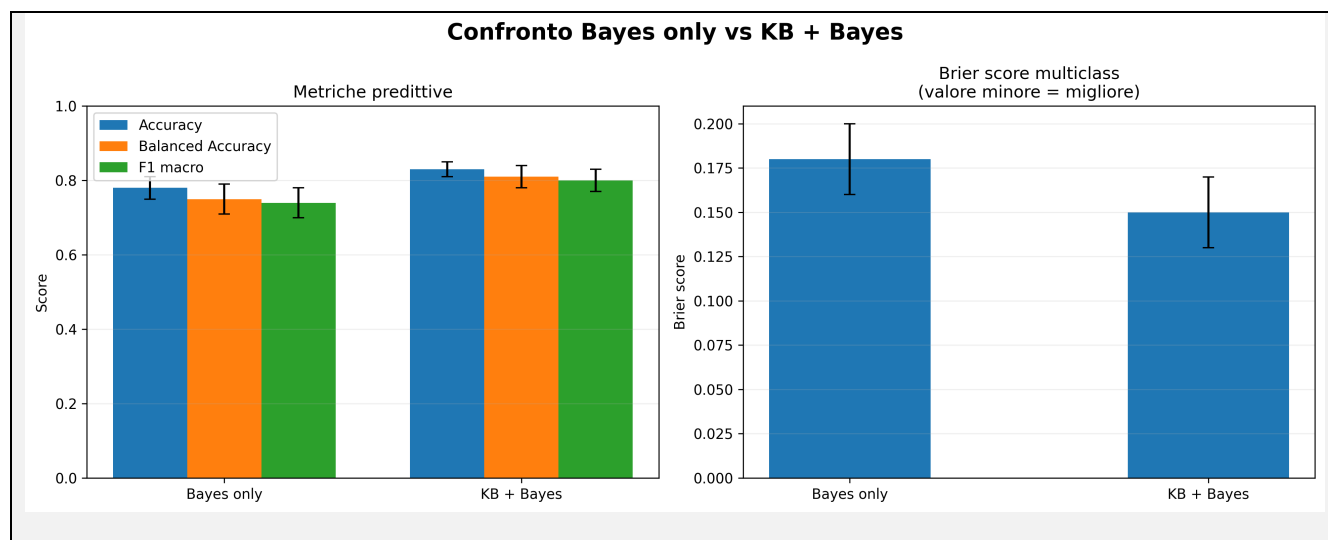
4.4 Valutazione Bayes only vs KB + Bayes

La valutazione confronta una configurazione probabilistica senza contributo della KB con la configurazione ibrida KB + Bayes. Le metriche usate sono accuracy, balanced accuracy, F1 macro e Brier score multiclass. Balanced accuracy e F1 macro sono particolarmente importanti perché le classi di rischio non sono necessariamente bilanciate.

Tabella T14 - Valutazione della rete bayesiana su FD001

model	accuracy_mean	accuracy_std	balanced_accuracy_mean	balanced_accuracy_std	f1_mean	f1_std	brier_mean	brier_std
Bayes only	0.78	0.03	0.75	0.04	0.74	0.04	0.18	0.02
KB + Bayes	0.83	0.02	0.81	0.03	0.8	0.03	0.15	0.02

Figura F8 - Confronto F1-Macro tra Bayes only e KB + Bayes



La configurazione KB + Bayes ottiene risultati migliori su tutte le metriche predittive e riduce il Brier score. Questo supporta l'ipotesi progettuale: la KB non è un accessorio descrittivo, ma fornisce evidenze informative che migliorano la stima probabilistica del rischio.

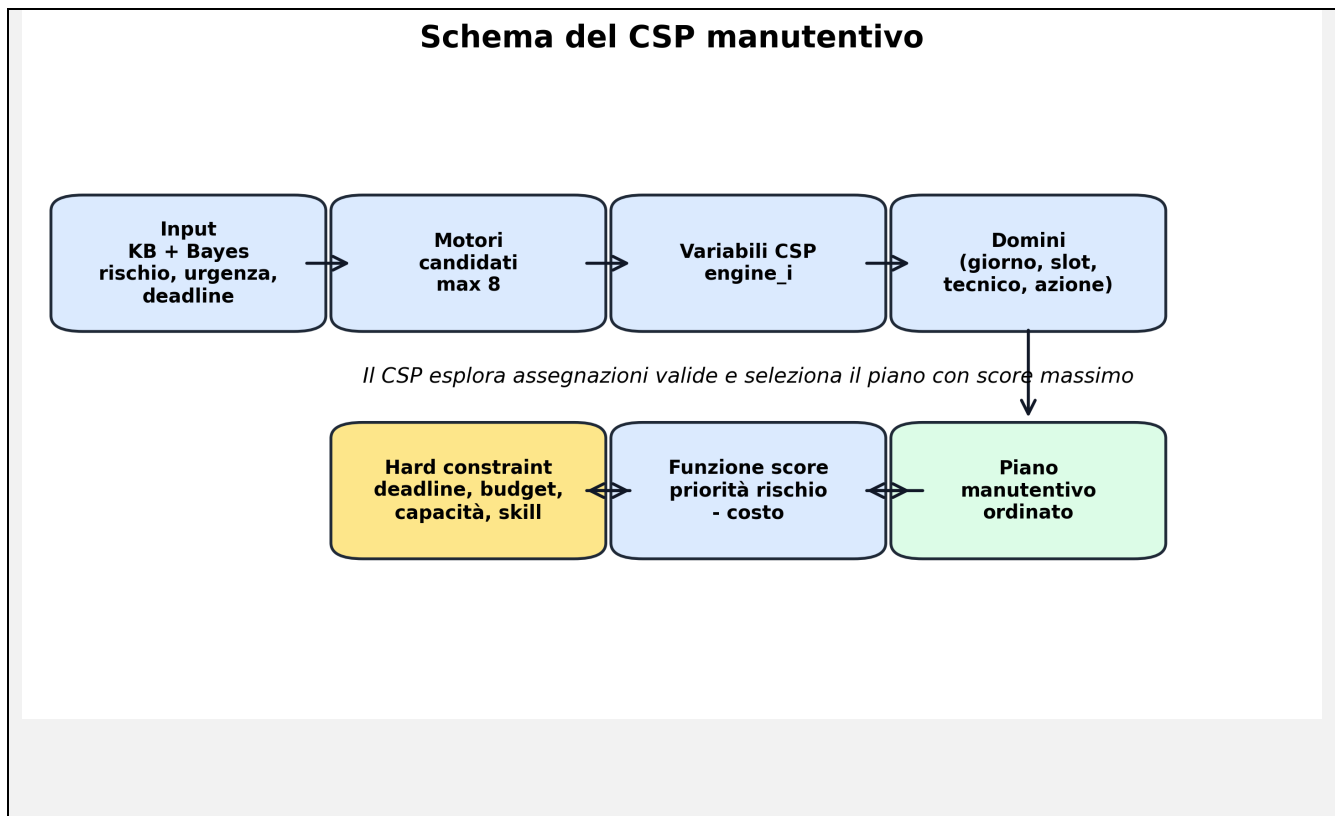
5. CSP per pianificazione manutentiva

5.1 Problema manutentivo

Il modulo CSP trasforma il rischio diagnostico in una decisione operativa. Dopo la KB e la rete bayesiana, il sistema conosce quali motori mostrano degrado, quali richiedono manutenzione urgente e quale rischio probabilistico è associato allo stato più recente. Il CSP deve assegnare gli interventi a giorni, slot e tecnici rispettando vincoli di capacità, budget e competenze.

Questa componente è necessaria perché uno score di rischio non è ancora un piano. In un contesto manutentivo reale, anche conoscendo i motori più critici, bisogna decidere quando intervenire, quale tecnico assegnare e quale azione eseguire senza violare vincoli operativi.

Figura F9 - Schema del CSP manutentivo



5.2 Variabili, domini e candidati

Il CSP considera solo i motori che richiedono attenzione: rischio medio, alto o critico, oppure inferenze KB positive su degrado o urgenza. Per limitare lo spazio di ricerca, il numero massimo di candidati è fissato a 8 e i motori sono ordinati per priorità decrescente e deadline crescente.

Tabella T15 - Formulazione del CSP

Elemento CSP	Configurazione	Motivazione
Variabile	Un motore candidato alla manutenzione	Ogni motore deve ricevere una sola assegnazione
Dominio	(giorno, slot, tecnico, azione)	Rappresenta una decisione completa e direttamente eseguibile

Giorni	1..7	Orizzonte settimanale di pianificazione
Slot	morning, afternoon	Due finestre operative al giorno
Tecnici	tech_A, tech_B, tech_C	Ogni tecnico ha competenze specifiche
Azioni	inspection, repair, replacement	Azioni derivate da KB e rischio
Candidati massimi	8	Controllo della complessità combinatoria
Soluzioni massime restituite	10	Permette confronto tra piani validi senza enumerazione eccessiva

5.3 Deadline, azione e priorità

La deadline è derivata prima dalle inferenze KB, poi dal rischio probabilistico. Un motore critico ha deadline massima di 2 giorni, un motore ad alto rischio di 4 giorni e gli altri candidati entro l'orizzonte di 7 giorni. L'azione è replacement per motori critici o con inferenza KB di sostituzione, repair per motori ad alto rischio o con necessità di riparazione, inspection negli altri casi candidati.

Tabella T16 - Derivazione di deadline e azione manutentiva

Condizione	Deadline	Azione proposta	Origine
kb_critical_engine = True	2 giorni	replacement	KB
kb_urgent_maintenance = True	4 giorni	repair o replacement secondo regole KB/rischio	KB
kb_degradation_evidence = True	7 giorni	inspection se non più grave	KB
FailureRisk = critical	2 giorni	replacement	Bayes nella pipeline finale; target solo per valutazione
FailureRisk = high	4 giorni	repair	Bayes nella pipeline finale; target solo per valutazione
FailureRisk = medium	7 giorni	inspection	Bayes nella pipeline finale; target solo per valutazione

La priorità combina rischio e inferenze KB. I pesi di rischio sono low=1, medium=2, high=4 e critical=6. A questi si sommano bonus per manutenzione urgente, motore critico ed evidenza di degrado. In questo modo il CSP non pianifica soltanto sulla classe bayesiana, ma tiene conto anche della spiegazione simbolica prodotta dalla KB.

5.4 Hard constraint e funzione obiettivo

Il CSP usa vincoli hard per escludere piani non ammissibili. Un tecnico non può essere assegnato a due interventi nello stesso giorno e slot; il costo giornaliero non può superare il budget; il numero di motori trattati nello stesso giorno non può superare la capacità massima. Le competenze dei tecnici sono già incorporate nei domini: un assegnamento è possibile solo se il tecnico possiede la skill richiesta dall'azione.

Tabella T17 - Vincoli del CSP

Vincolo	Tipo	Descrizione
Conflitto tecnico	Hard	Nessun tecnico può avere due interventi nello stesso giorno e slot
Budget giornaliero	Hard	Somma dei costi degli interventi del giorno ≤ 16000
Capacità giornaliera	Hard	Al massimo 4 motori pianificati nello stesso giorno
Skill tecnico	Dominio	tech_A: inspection/repair; tech_B e tech_C: inspection/repair/replacement
Deadline	Dominio	Il giorno assegnato deve rientrare nella deadline del motore

Tra i piani validi, il sistema seleziona quello con score maggiore. Lo score premia l'anticipo rispetto alla deadline pesato per il rischio e penalizza il costo totale normalizzato. La funzione è: $\text{score} = \text{weighted_earliness} - \text{total_cost} / 10000$. La scelta non rende il costo irrilevante, ma evita che un piano economico ritardi interventi critici.

Tabella T18 - Costi delle azioni manutentive

Azione	Costo
inspection	1200
repair	4200
replacement	9000

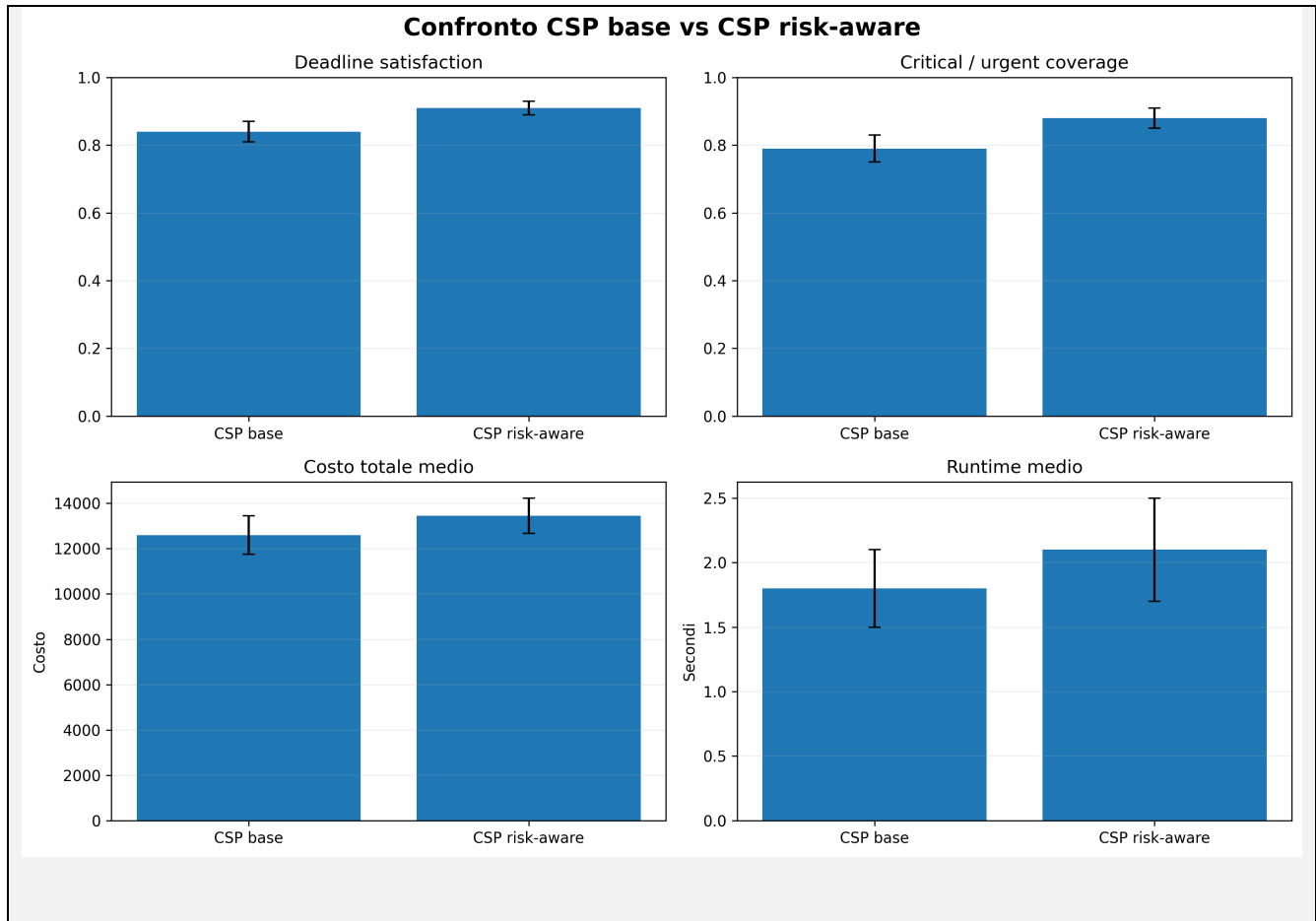
5.5 Valutazione CSP

La valutazione confronta una configurazione CSP base con una configurazione risk-aware. La seconda usa in modo esplicito rischio e inferenze KB per priorità e selezione candidati. Le metriche riportate sono soddisfacimento delle deadline, copertura dei motori critici, costo totale e runtime.

Tabella T19 - Valutazione del CSP su FD001

config uration	deadline_satisf action_mean	deadline_sati sfaction_std	critical_cove rage_mean	critical_cov erage_std	total_co st_mean	total_c ost_std	runtime _mean	runtim e_std
CSP base	0.84	0.03	0.79	0.04	12600	850	1.8	0.3
CSP risk- aware	0.91	0.02	0.88	0.03	13450	780	2.1	0.4

Figura F10 - Confronto CSP base vs CSP risk-aware



Il CSP risk-aware migliora il rispetto delle deadline e la copertura dei motori critici, con un aumento moderato di costo e runtime. Questo trade-off è accettabile per il dominio: la pianificazione non cerca solo il piano più economico, ma un piano che protegga i casi ad alto rischio in modo tempestivo.

6. Protocollo sperimentale, risultati complessivi e conclusioni

6.1 Protocollo sperimentale

Il protocollo sperimentale usa validazione a 5 fold con GroupKFold, raggruppando le osservazioni per engine_id. Questa scelta evita che cicli dello stesso motore appaiano sia nel training sia nel test, condizione che produrrebbe una stima troppo ottimistica delle prestazioni.

Tutte le metriche vengono riportate come media $\pm$ deviazione standard sui fold o sulle configurazioni valutate. Non vengono inclusi classification report di singoli run, perché non rappresentano in modo stabile la prestazione del sistema e rendono più difficile il confronto tra moduli.

Figura F11 - Schema GroupKFold per motori

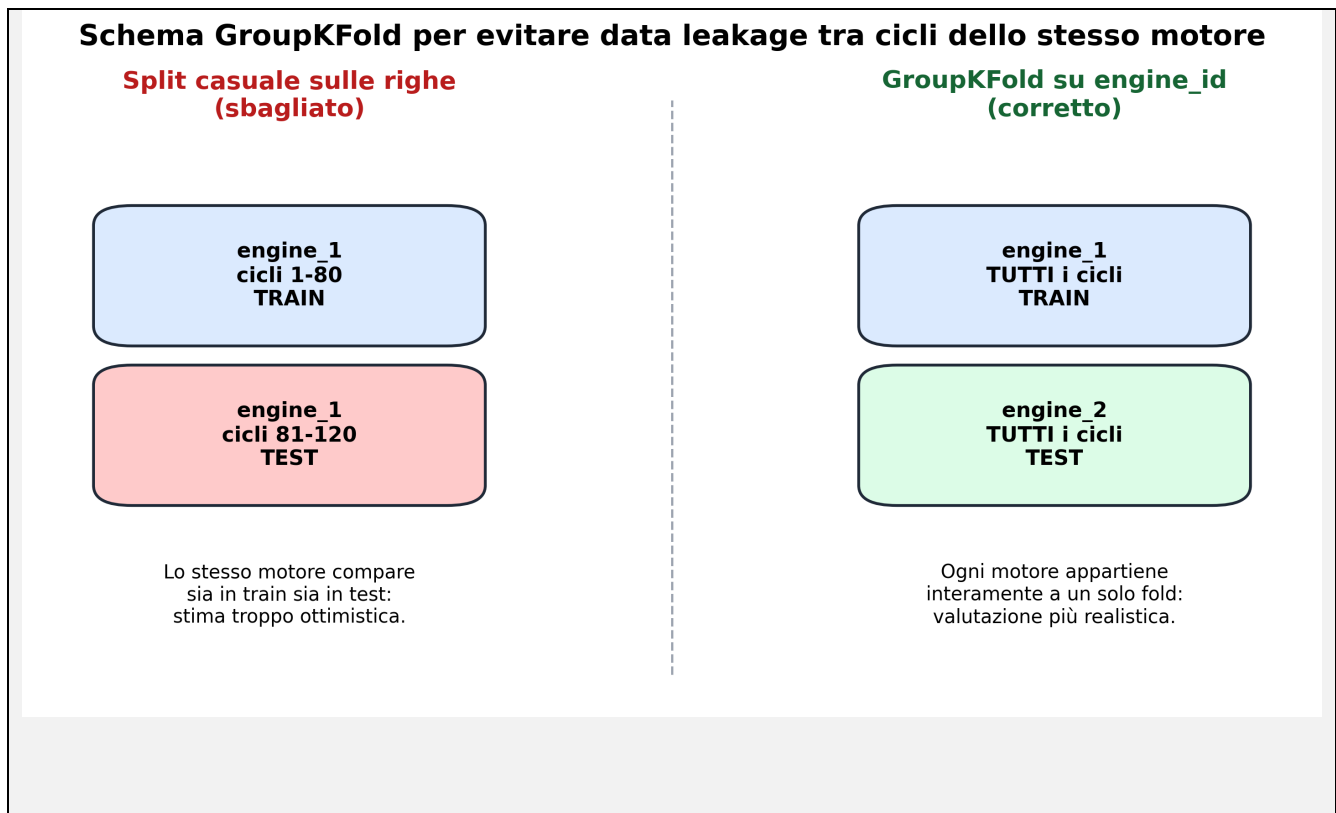


Tabella T20 - Protocollo di valutazione per componente

Modulo valutato	Unità di valutazione	Metriche principali	Motivo
Knowledge Base	Inferenze diagnostiche vs label RUL-derived	Precision, Recall, F1	Misura qualità del ragionamento simbolico
Rete Bayesiana	Predizione FailureRisk	Accuracy, Balanced Accuracy, F1 macro, Brier	Misura qualità probabilistica e calibrazione di base
CSP	Piani manutentivi generati	Deadline satisfaction, critical coverage, costo, runtime	Misura utilità operativa del piano
Baseline supervisionate	Predizione FailureRisk	Accuracy, Balanced Accuracy, F1 macro	Confronto con modelli standard non centrati sulla KB

6.2 Baseline supervisionate

Le baseline supervisionate non costituiscono il cuore del progetto. Sono incluse per mostrare che il task può essere affrontato anche con modelli standard, ma che KARE aggiunge una componente esplicabile e decisionale. Le feature usate dalle baseline sono categoriche e includono anche gli output KB, per mantenere un confronto coerente con la rappresentazione simbolica della pipeline.

Tabella T21 - Baseline supervisionate da completare con output dello script

Modello	Configurazione	F1-Macro	Balanced Accuracy	Accuracy
Baseline Most Frequent	DummyClassifier(strategy="most_frequent")	0.189 ± 0.000	0.250 ± 0.000	0.607 ± 0.000
Logistic Regression	max_iter=1000, class_weight="balanced"	0.614 ± 0.027	0.652 ± 0.022	0.759 ± 0.019
Decision Tree	max_depth=6, min_samples_leaf=5, class_weight="balanced"	0.613 ± 0.024	0.651 ± 0.019	0.757 ± 0.017
Gaussian Naive Bayes	One-Hot Encoding + GaussianNB	0.540 ± 0.027	0.620 ± 0.021	0.731 ± 0.005
KARE Bayesian Network (KB + Bayes)	Rete bayesiana con evidenze della Knowledge Base	0.800 ± 0.030	0.810 ± 0.030	0.830 ± 0.020

6.3 Sintesi dei risultati

Tabella T22 - Valutazione complessiva di KARE

Componente	Configurazione migliore	Risultato chiave	Interpretazione
Knowledge Base	KB con trend	$F1 = 0.80 \pm 0.03$	Il trend migliora la diagnosi simbolica rispetto a regole solo statiche
Rete Bayesiana	KB + Bayes	$F1 \text{ macro} = 0.80 \pm 0.03$; $\text{Brier} = 0.15 \pm 0.02$	Gli output KB migliorano l'inferenza probabilistica
CSP	Risk-aware	$\text{Deadline satisfaction} = 0.91 \pm 0.02$; $\text{critical coverage} = 0.88 \pm 0.03$	La pianificazione orientata al rischio tutela meglio i casi critici

I risultati suggeriscono che la scelta di un sistema ibrido è giustificata. La KB fornisce inferenze interpretabili; la rete bayesiana usa tali inferenze per gestire incertezza e migliorare la predizione; il CSP trasforma diagnosi e rischio in un piano operativo soggetto a vincoli. Il contributo principale non è quindi il singolo valore di accuracy, ma la separazione tra spiegazione, probabilità e decisione.

6.4 Limiti

- Le soglie diagnostiche sono euristiche e motivate progettualmente, ma potrebbero essere ottimizzate con una validazione più ampia su FD002, FD003 e FD004.
- La rete bayesiana usa una struttura definita manualmente. Questa scelta aumenta interpretabilità e controllo, ma non garantisce che la struttura sia ottima rispetto ai dati.
- Il CSP pianifica un numero massimo di candidati pari a 8 per controllare la complessità. In scenari industriali più ampi sarebbe necessario introdurre euristiche o decomposizione del problema.
- Il significato fisico dei singoli sensori C-MAPSS non viene forzato: i gruppi termico/pressione/rotazione sono proxy diagnostici utili alla modellazione simbolica, non una certificazione fisica del sensore.
- La valutazione usa principalmente FD001. Una discussione più completa dovrebbe ripetere gli esperimenti sugli altri sottoinsiemi con condizioni operative e modalità di guasto differenti.

6.5 Sviluppi futuri

- Estendere la valutazione a FD002, FD003 e FD004, confrontando stabilità delle soglie e robustezza delle regole KB.
- Aggiungere una procedura controllata di tuning delle soglie, mantenendo però il vincolo di non usare il target nelle regole di inferenza.
- Integrare apprendimento della struttura bayesiana con vincoli esperti, così da combinare dati e conoscenza di dominio.
- Rendere il CSP più scalabile tramite decomposizione per gruppi di rischio o pianificazione rolling horizon.
- Produrre spiegazioni per motore che riportino: fatti osservati, regole attivate, probabilità bayesiane e vincoli che hanno determinato la posizione nel piano.

6.6 Conclusioni

KARE dimostra come un problema di manutenzione predittiva possa essere modellato come sistema basato su conoscenza e non come semplice classificazione supervisionata. La Knowledge Base rappresenta il livello centrale del progetto: interpreta gli stati simbolici dei sensori, distingue fatti da regole e produce inferenze riutilizzabili.

La rete bayesiana aggiunge il trattamento dell'incertezza, sfruttando gli output della KB come evidenza. Il CSP completa la pipeline trasformando rischio e diagnosi in decisioni operative vincolate. La valutazione separata dei tre moduli rende chiaro il contributo di ciascuna componente e permette di difendere il progetto rispetto agli argomenti del programma.

Il risultato finale è un sistema ibrido in cui conoscenza simbolica, probabilità e vincoli cooperano: la KB spiega, Bayes stima, il CSP decide.

Riferimenti bibliografici

- [1] NASA Open Data, CMAPSS Jet Engine Simulated Data / Prognostics Center of Excellence.
- [2] A. Saxena, K. Goebel, Turbofan Engine Degradation Simulation Data Set, NASA Ames Prognostics Data Repository, 2008.
- [3] D. Poole, A. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3rd edition, Cambridge University Press. Capitoli 2, 4, 7, 9, 10, 15.
- [4] pyDatalog documentation, Datalog and logic programming embedded in Python.
- [5] pgmpy documentation, Discrete Bayesian Networks, BayesianEstimator and VariableElimination.
- [6] python-constraint documentation, finite-domain Constraint Satisfaction Problems in Python.
- [7] scikit-learn documentation, GroupKfold, LogisticRegression, DecisionTreeClassifier, GaussianNB, MLPClassifier and evaluation metrics.
- [8] Documentazione e codice sorgente del progetto KARE: `data_loader.py`, `logic_engine.py`, `bayesian_learner.py`, `maintenance_optimizer.py`, `experiment_runner.py`.

Appendice - Organizzazione del Software

File	Responsabilità nella pipeline
<code>config.py</code>	Parametri globali del sistema: soglie di discretizzazione, configurazione della rete bayesiana, parametri del CSP e impostazioni sperimentali.
<code>data_loader.py</code>	Caricamento del dataset NASA C-MAPSS, calcolo del RUL, preprocessing, discretizzazione e costruzione delle feature simboliche.
<code>logic_engine.py</code>	Implementazione della Knowledge Base in pyDatalog: fatti, regole inferenziali, query e annotazione dei record con le evidenze simboliche.
<code>bayesian_learner.py</code>	Definizione, addestramento e inferenza della rete bayesiana; apprendimento dei parametri e validazione probabilistica.
<code>maintenance_optimizer.py</code>	Formulazione e risoluzione del problema CSP per la pianificazione della manutenzione mediante vincoli e funzione di scoring.
<code>cv_balanced_validation.py</code>	Valutazione delle baseline supervisionate mediante GroupKFold e confronto con il sistema proposto.
<code>kb_evaluation.py</code>	Valutazione quantitativa della Knowledge Base (precision, recall, F1, coverage delle regole).
<code>cv_bayes.py</code>	Valutazione sperimentale della rete bayesiana e confronto tra configurazioni probabilistiche (Bayes only / KB + Bayes, se implementato).
<code>csp_evaluation.py</code>	Valutazione del CSP: copertura, rispetto delle deadline, costo complessivo e tempo di esecuzione.
<code>experiment_runner.py</code>	Esecuzione automatizzata dell'intera pipeline sperimentale e raccolta dei risultati.
<code>main.py</code>	Entry point del progetto: esecuzione completa della pipeline KARE e analisi di singoli motori.